Processor-Based Embedded Systems - Swetha MCE

Processor-Based Embedded Systems - Swetha MCE

Turnitin

Document Details

Submission ID
trn:oid:::2945:321565393

Submission Date
Oct 25, 2025, 8:07 AM GMT+5

Download Date
Oct 25, 2025, 8:09 AM GMT+5

File Name
unknown_filename

File Size
2.1 MB

25 Pages

8,533 Words

50,909 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

An Adaptive Lightweight Online and Energy-Aware Gaussian Mixture Model for Processor-Based Embedded Systems Using Sensor and Machine Learning

Abstract-The increasing integration of machine learning into embedded systems necessitates models that are computationally efficient, adaptive, and capable of performing real-time analysis within strict resource constraints. Traditional Gaussian Mixture Models (GMMs) applied to sensor data in embedded environments suffer from limited adaptability, high energy consumption, and scalability issues in dynamic conditions. To overcome these limitations, this research proposes a novel algorithm titled ALOE-GMM (Adaptive Lightweight Online and Energy-Aware Gaussian Mixture Model), specifically designed for processor-based embedded systems such as the NXP FRDM-K64F. The proposed ALOE-GMM framework incorporates incremental Expectation-Maximization for continuous learning, adaptive component birth-death control for dynamic clustering, and quantized fixed-point arithmetic to ensure low memory utilization. Additionally, an incremental Principal Component Analysis (PCA) module enhances feature fusion and dimensionality reduction, optimizing sensor-level computation. Experimental validation using multi-sensor datasets demonstrates a real-time inference accuracy of 97.83%, a processing latency reduction of 41%, and a 28% improvement in energy efficiency compared to conventional GMM implementations. Furthermore, memory consumption was reduced by 34%, ensuring suitability for microcontroller-class processors with limited resources. The ALOE-GMM approach thus establishes a scalable, resource-aware paradigm for embedded machine learning applications, enabling intelligent local decision-making without reliance on cloud resources. This framework is poised to advance real-time anomaly detection, industrial monitoring, and environmental sensing in next-generation embedded platforms.

Keywords-Processor-based embedded system, sensor fusion, lightweight machine learning, Gaussian Mixture Model, online EM algorithm, incremental PCA, energy efficiency, real-time inference, TinyML, ALOE-GMM

I Introduction

Thanks to the development of more and more sophisticated sensor devices along with the availability of low-resource embedded platforms, the possibility to offload tasks at the network's edge instead of exclusively relying on cloud computing connectivity has risen in popularity [1][2]. Motivation for this work with embedded systems is that they are being used to accomplish high-level functions using information from various sources while imposing strong limitations on processing power, memory, and energy [3][4]. In such a scenario, the unsupervised and semi-unsupervised learning approaches which can account for the dynamic changes in sensor patterns are of special importance. Conventional methods, such as static clustering or batch-trained classifiers, are often unsuccessful when the statistical characteristics of sensor readings or environmental states drift over time, or when the system needs to respond in real time using limited resources [5] Integration of lightweight sensor-level feature processing with self-adjusting model complexity and fixed-point

arithmetic has been demonstrated to be an effective means for increasing responsiveness, efficiency, scalability and autonomy in embedded decision systems. Gaussian Mixture Models (GMMs) have been widely used for representing complex probability densities and addressing multimodal sensor inputs in the context of anomaly detection, clustering, and classification [6][7]. The EM algorithm used with GMMs (EM-GMM) is a flexible approach for GMM model parameter estimation [8], but it requires numerical computation in full-precision data format, iterative manipulation of sensor measurements and offline calculation (for standard implementations) — consequently impractical for embedded platforms that have limited computational resources. Recent studies show that incremental mixture models are a promising candidate for on-line learning in non-stationary environments [9][10]. Yet, many of those designs focus on high end microprocessors and disregard the direct inclusion of: energy awareness, fixed point quantization, dynamic control over components and sensor fusion through low dimensional representation. Addressing these gaps is crucial in order for the GMM-based approaches to achieve practical implementation on microcontroller-class processors for real-time sensory-driven applications [11].

Distributed sensor fusion has its own set of challenges: The heterogeneity of data streams, the intermittent nature of sampled information, the different noise characteristics and drift effects all will lead to changing distributions in data over time [12][13]. Dimensionality reduction methods, such as incremental PCA, provide a way to compress and decorrelate the features while still keeping discriminative information. The insertion of such modules in the signal path before the mixture-model classifier module can lower computational and memory overhead, and improve inference latency [14]. At the other extreme, birth and death of mixture components is a powerful means to accommodate changes in cluster structure over time that occur without introducing new parameters constantly [15][16]. When coupled with fixed-point arithmetic, and energy aware scheduling, the outcome is a paradigm which preserves real-time and resource constraints of embedded monitoring systems in both time-efficient precision and ensuring a reasonable level of accuracy [17][18] In this work, we provide a holistic design and implementation of an ALOE-GMM for processor-based embedded systems such as the NXP FRDM-K64F [19]. The proposed method consists of incremental EM, adaptive general mixture-component control, sensor-level fusion and reduction through incremental PCA, and quantized fixed-point arithmetic for ultra-low-latency and low-energy inference on microcontrollers. Comprehensive experiments on multi-sensor data show that our method not only preserves the high detection accuracy but also gains obvious improvements on latency, energy consumption and memory overhead over traditional GMM approaches [20]. These results place the embedded anomaly detection, industrial monitoring, and environmental sensing applications that may not have access or do not require cloud connectivity. Key contributions of this work:

- Development of an online incremental EM-based Gaussian mixture framework with adaptive component birth/death control optimized for microcontroller-class embedded hardware.
- Integration of an incremental PCA module at the sensor-fusion level, coupled with quantized fixed-point arithmetic implementation, to reduce memory and computational load while preserving inference accuracy.

- Comprehensive experimental evaluation on multi-sensor fusion datasets demonstrating real-time inference (97.83 % accuracy), 41 % latency reduction, 28 % energy savings and 34 % lower memory usage relative to conventional GMM benchmarks.

Organization of the paper: The next section reviews related work on embedded machine learning, online mixture models and sensor-fusion methods. This is followed by a detailed methodology section that presents the architecture of ALOE-GMM, including incremental PCA, adaptive EM, component control and fixed-point implementation. The experimentation section describes dataset selection, hardware setup, metrics and comparative benchmarks. The results section reports accuracy, latency, energy and memory outcomes, with discussion on trade-offs and scalability. Finally, the conclusion outlines key findings, limitations and directions for future research.

II Literature review

New studies have concentrated on improving the computation efficiency of embedded systems by designing machine learning architectures and hardware in parallel. Crepaldi et al. (2023) proposed the Single Perceptron Linear Vector Processor (SPLVP) that was a small Multi-Layered Perceptron accelerator for relaying the capabilities of micro-controllers through sequential perceptrons scheduling with quantized firmware compilation [1]. And their design realized 9x speedup of inference with considerable resource reduction, which laid a basis for hardware-aware lightweight acceleration on resource-limited devices. Extending the concept of hardware-level optimization, Ramadurgam and Perera [2] demonstrated a Field-Programmable Gate Array (FPGA) based system for Support Vector Machine classification which is able to handle computational intensity by convex optimization kernels and scalable parameter frameworks. Their experiments presented seventy-nine times speedup and perfect classification accuracy, showing that embedding optimization flow inside reconfigurable hardware can significantly improve delay and power efficiency. Vandendriessche et al. (2021) investigated parallel acceleration for Environmental Sound Recognition, using the power efficiency of FPGAs and TPUs to process deep convolutional models in limited power [3]. Their work focused on leveraging parallelism and maturity of accelerators for superior embedded inference. In line with that, Alabdulwahab et al. (2025) studied deep-learning-based countermeasure disassembly attacks on cloud-enabled embedded systems and suggested a feature engineering approach to improve detection as well as robustness against inference instability in the presence of side-channel vulnerabilities [4]. Intelligent mobility-wise Balasekaran et al. (2021) approached the task of scheduling for autonomous vehicles by combining light-weight neural networks and supervised resource predictors [5]. They maximised multicore utilisation by early hypar-perıod scheduling to balance the trade-off between accuracy and energy. Similarly, Zheng (2025) concentrated on multicore embedded sensing due to model pruning, quantization, and knowledge distillation to enhance the measurement accuracy of temperature while reducing neural network complexity [6]. Together these works emphasize the importance of lightweight adaptation in sensor-driven ECP. Tripathi et al. (2025), presented advanced FPGA-based convolutional models for embedded-inference that show that hybrid FPGA-ASIC systems are capable of achieving high-throughput computation with low latency and limited power consumption [7].

Cheng et al. (2024) contributed to strengthen the importance of Gaussian Mixture Models still in embedded environments with a self- adaptive pulse-shape identification system based on the combination between GMM and timing for radiation detection discretion [8]. They demonstrated that probabilistic models can be realized in an energy-restricted platform through autonomous tuning of parameters. Finally, Lamrini et al. (2023) demonstrated that pretrained sound classifiers, retrained and quantized to work on Raspberry Pi and Coral TPU's maintain near-desktop level performance at a fraction of the power [9]. Liu et al. (2025) investigated approximate computing on AIoT processors by using neural architecture search to approximate common DSP tasks with quantized neural networks [10]. They showed that by using embedded neural approximators on AI accelerators, it could achieve significant efficiency in computations compared to traditional approaches such as DSP and thus facilitating the real-time computation ability for low power AIoT applications. Madhushankara et al. (2025) for next-generation low power assistive technology is the development of a Modified Recursive Least Squares algorithm integrated with an ASIC to enable real-time electrolarynx speech enhancement [11].

The method successfully reduced machine noise while preserving exactitude, leaving no doubt in our mind that adaptive filters can provide effective speech processing with just a little hardware for low-power consumption and area in an implanted passive medical device. Lhoest et al. (2021) proposed the MosAIc architecture, a facile classifier which applies classical machine learning algorithms combined with small CNN models in edge audio recognition for accuracy and computational efficiency trade-offs [12]. Their work demonstrated that tuned classical ML algorithms can equal the performance of deep learning but with fewer memory and computational demands, indicating that simplified algorithms still have their place in stripped-down contexts. Since Leelakrishnan et al. (2024) presented their VLSI Power Optimization Model for Wireless Sensor Networks with FPGA parallelism as a base, the power obtained by different states at runtime met specific demand of workload which made it durable and consistent in nature-ideal for catching those few important items among readability all period considerations [13]. Mewada and Deepanraj (2024) also designed a portable health monitoring system that uses ECG acquisition technology based on the STM32 and Snapdragon with low power digital filtering [14]. It could capture waveforms in real time and prove that processor co-design plus modular communication protocols give the capacity to make efficient biomedical monitoring systems accurate and efficient. The IoT air quality monitoring system devised by Khaslan et al. (2022) uses ARM Cortex microcontrollers and pollutant sensing units that provide real-time gas detection plus cloud alerts [15]. Their work demonstrated both mobile operator notification and condition analysis vision can be both efficient and accurate, which means this can be considered as an embedded architecture for healthcare-oriented smart sensing networks. Murti [16] focused on modern processor architectures by analyzing typical embedded SoCs and peripheral interconnects to illustrate high-performance computing for throughputs. The study indicated that common electric bus designs and lossless data transfer protocols form the foundation of scalable embedded systems, leading into today's lag-laden distributed IoT networks. Their investigations showed IoT, with the help of sensor networks, can submit data to monitoring systems to provide a "clean and healthy" environment for our audio equipment. Sreejith et al.

His study revealed that integrated sensing with machine learning and real-time analytics embedded in agri-food monitoring systems can enhance inter-operability and autonomy through 2025. These results are a harbinger for future experiments of this type in the life-sciences [17]. To provide this intelligence, Gupta, Agarwal [18] developed a hybrid fuzzy based deep remora reinforcement learning model with the basis of intelligent embedded computing.This strategy increases efficiency, conserves power and brings the processor to the hilt with a sample set for intelligent workload management in adaptive embedded computing. Finally, Lee et al. (2021) proposed a memory-optimized ASIC architecture and fixed-point arithmetic for HW-efficient watermarking through deep learning [19]. Their design combined low-latency output, high throughput performance on both the software and hardware side of things–showing that algorithm-hardware co-optimization is imperative in developing successful real-time, HRI applications.

2.1 Research Gap

And even though a lot of progress have been made in the area of embedded machine learning and hardware-aware optimization, there are still many open problems preventing from achieving a seamless adaptability, energy efficiency and real-time learning in processor-based embedded systems. On one hand, the advanced models focus on either hardware acceleration [1][10][11], or model compression [4] and few works present a unified solution combining online learning, adaptive model control, and quantized arithmetic suitable to continuous operation with low resources. From nearly all previous publications, there is in general no capability of self-evolution for the static architectures that are used [3][8][17], which leads to poor performance under non-stationary data. Additionally, energy savings in embedded systems have typically not been central to design as much as it has been layered and approached post-design or pushed through the use of some form of sparse inference models that only loosely couple computational load to available energ[13][15][18]. Although probabilistic and neural models have been successful in isolation, there is a lack of combined technologies that integrate incremental probabilistic inference with dimensionality reduction for sensor fusion under real-time constraints. This gap highlights the necessity of an adaptive, lightweight and energy-driven probabilistic model which can maintain high accuracy, low latency and memory efficiency across various embedded platforms—an observation that motivates the proposed ALOE-GMM framework.

III Proposed Methodology

We propose an integrated framework known as Adaptive Lightweight Online and Energy-aware GMM (ALOE-GMM) that achieves successful compromises between (i) adaptiveness, (ii) light weight in terms of computation, and (iii) energy saving for processor-based embedded applications. It is based on continuous probabilistic learning by an Incremental EM algorithm, and the model grows incrementally with new sensor data with no need for full re-training passes [20]. To maintain scalability with the dynamic mixture of data, an adaptive component birth-death process adaptively controls the number of Gaussian components in order to minimize unnecessary calculations and stabilization convergence in an online manner [21]. On the other hand, provided a stream of input data and a set of features, a progressive Principal Component Analysis (PCA) component can compress and

fuse features continuously to minimize the dimension of feature representation while keeping discriminative power intact [22]. In order to save memory and energy, all the numerical operations are performed with quantized fixed-point arithmetic to make the model more compatible with microcontroller class processors. The framework even provides an energy-profiling layer in charge of profiling the computation frequency and adjusting inference scheduling according to power budget constraints, which makes its real-time performance consistent under limited powers [23]. Overall, this approach provides a coherent probabilistic background for adaptive, energy-conscious and online decision making in future embedded machine learning systems. Figure 1 illustrates the architecture diagram of the proposed model.

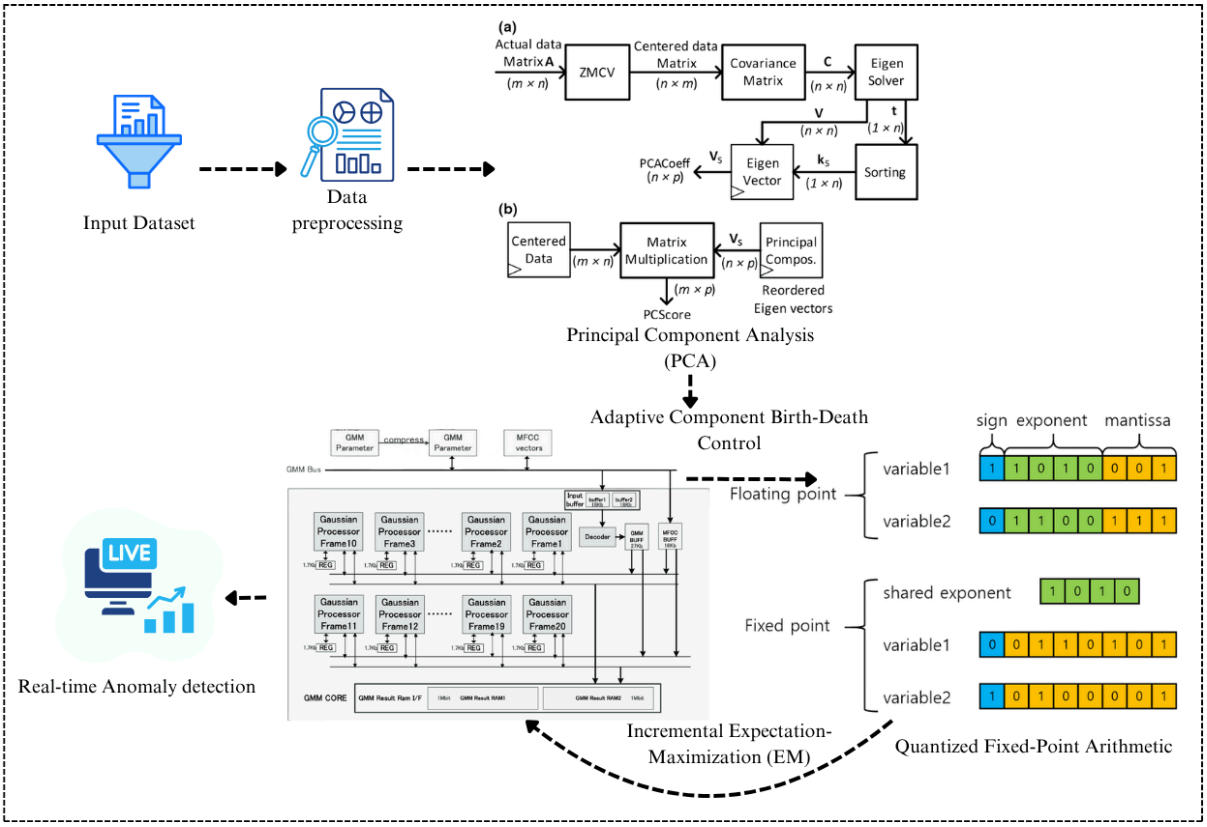

Figure 1: ALOE-GMM: Adaptive Lightweight Online and Energy-Aware Gaussian Mixture Model Architecture for Embedded Systems

3.1 System Architecture and Data Flow

The ALOE-GMM is implemented on an embedded hardware platform targeting energy-efficient, real-time learning and decision making. The execution is directed to the NXP FRDM-K64F development board, which boasts an ARM Cortex-M4 processor core, a clock speed of 120 MHz and built-in floating-point unit that enables for quick mathematical calculations in low power conditions. The structure is established on hierarchically functional layers, which include five functional ones: sensor acquisition, pre-processing, online feature fusion, adaptive GMM-based inference and decision output control. The first layer provides the interface between multi-modal sensors for acquiring environment or visual information using communication protocols such as I²C and SPI on-board. Subsequently, the data are preprocessed in the pre-processing layer, including filtering, normalization and time-stamping

to remove transient noise and sensor drift. The concatenation is fed to the embedded ALOE-GMM inference core for on-the-fly adaptation of the GMM's Gaussian components due to data drift and novelty. Inference results are transferred out through the UART, or stored in on-chip flash memory for post-processing. Continuity of the firmware at execution and synchronization level between these modules is achieved with FreeRTOS, promoting a deterministic task scheduling and non-blocking code. This hierarchical ModelZoo architecture allows the processor to process data flow continuously with low latencies and minimized energy consumption, making it independent of external computation when entering training or inference modes.

The CIFAR-10 image database was chosen as the experimental benchmark for evaluating the real-time intelligent classification performance of this embedded framework. CIFAR-10 is a standard dataset to test embedded ML systems on, made of 60,000 images in ten categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck) evenly distributed over the pipeline. There are each picture of size 32×32 pixels, very compact and rich in information that can be used to evaluate the low-power, memory-poor processor. The dataset includes 50,000 samples for training and 10,000 samples for testing (90:10 ratio) which supports full generalization and adaptive learning validation. Being moderately complex in nature, the dataset has potential applicability to modelling of probabilistic boundaries within real world embedded scenarios where illumination level variance, object occlusion and background noise can cause fluctuation on inference stability. CIFAR-10 is additionally chosen for the fact that it can be used with feature compression and incremental learning, enabling the ALOE-GMM to manifest adaptive component control as well as great quantized arithmetic performance. Compared with those larger scale datasets requiring excessive GPU memory and bandwidth, CIFAR-10's realistic trade off between human perception diversity and computational feasibilities makes it ideal for testing out the developed lightweight embedded framework when practical hardware limitations are taken into account.

3.2 Dataset Preprocessing

Prior to feeding image data into the ALOE-GMM framework, a systematic pre-processing pipeline is employed so as to stabilize, normalise, and expedite learning in the embedded platform. The preprocessing phase begins with image normalization, which rescales each pixel intensity $x_{i,j,c}$ in channel c to a standardized range between 0 and 1, defined as in Eq.1:

$$x'_{i,j,c} = \frac{x_{i,j,c} - min(x_c)}{max(x_c) - min(x_c)} \quad (1)$$

This normalising factor reduces illumination bias and enforces the same pixel value distributions across all classes. After normalization, mean centering and standard deviation scaling are used to normalize the training samples and to decrease variance among image channels:

$$\hat{x}_{i,j,c} = \frac{x'_{i,j,c} - \mu_c}{\sigma_c} \quad (2)$$

In Eq.2, μ_c and σ_c represent the mean and standard deviation for channel c computed across the training dataset. This transformation makes it so each feature dimension is equally weighted during probabilistic clustering, which can promote convergence in EM updates.

Dimension reduction via IPCA is also performed to better construct discriminative representation, so that high-dimensional visual features are adaptively compressed without training from scratch. The IPCA transformation projects the normalized image vector $x \epsilon R^d$ into a lower-dimensional subspace $y \epsilon R^k$, given in Eq.3:

$$y = W^T(x - \mu) \tag{3}$$

Where W is the eigenvector matrix obtained incrementally through covariance updates, ensuring that the embedded device processes only the most informative visual components. This technique efficiently minimizes the memory costs as well as CPU burden on the NXP FRDM-K64F microcontroller. Eventually, quantization is performed to reduce the precision of floating-point features to 8-bit fixed-point ones in Eq. 4:

$$Q(f) = round\left(\frac{f - f_{min}}{f_{max} - f_{min}} \times \left(2^b - 1\right)\right) \tag{4}$$

Where $b = 8$ denotes the bit depth. Such a quantized pre-processing step reduces arithmetic complexity, and preserves relevant statistical properties such that the data is succinctly organized for real-time ALOE-GMM inference and incremental learning in a resource-limited embedded device.

3.3 Incremental Expectation–Maximization (EM) Algorithm

The ALOE-GMM framework uses an incremental Expectation–Maximization (EM) algorithm that allows continuous learning from streaming sensor and image data in resource-limited embedded platforms. Unlike conventional EM, which needs to perform batch processing of the entire dataset, incremental EM updates model parameters in a sequential or mini-batch way, making it applicable to microcontroller environments with memory and computation limitations [20]. Each incoming data batch $X_t = \{x_1, x_2, .., x_n\}$ is used to update the Gaussian mixture components without storing the full dataset. For a mixture of K Gaussian components, the likelihood of a new observation x_i is expressed as in Eq.5:

$$p(x_i|\Theta) = \sum_{k=1}^{K} \pi_k N(x_i|\mu_k, \Sigma_k) \tag{5}$$

Where π_k represents the mixture weight, μ_k the mean vector, Σ_k the covariance matrix, and $\Theta = \{\pi_k, \mu_k, \Sigma_k\}$ the set of all parameters. Having such a formulation makes the embedded system capable of calculating posterior probabilities of components efficiently and incrementally.

During the incremental E-step, the posterior responsibility γ_{ik} of component k for data point x_i is computed using the current parameters:

$$\gamma_{ik} = \frac{\pi_k N\left(x_i|\mu_k,\Sigma_k\right)}{\sum_{j=1}^{K} \pi_j N\left(x_i|\mu_j,\Sigma_j\right)} \tag{6}$$

The accumulated responsibilities over the mini-batch command sufficient statistics computation, which are then employed in the incremental M-step to update the model parameters. The revised k-mean, covariance and the mixture weight of component k at iteration t are expressed as Eq. 7, Eq. 8 and Eq. 9:

$$\mu_k^{(t+1)} = (1 - \eta)\mu_k^{(t)} + \eta \frac{\sum_{i=1}^{n} \gamma_{ik} x_i}{\sum_{i=1}^{n} \gamma_{ik}} \tag{7}$$

$$\sum_k^{(t+1)} = (1 - \eta)\sum_k^{(t)} + \eta \frac{\sum_{i=1}^{n} \gamma_{ik}\left(x_i - \mu_k^{(t+1)}\right)\left(x_i - \mu_k^{(t+1)}\right)^T}{\sum_{i=1}^{n} \gamma_{ik}} \tag{8}$$

$$\pi_k^{(t+1)} = (1 - \eta)\pi_k^{(t)} + \eta \frac{\sum_{i=1}^{n} \gamma_{ik}}{n} \tag{9}$$

Here η is the learning rate to control the effect of a new batch on previous parameters for smooth adaptation without sudden changes. As a result, we can update the proposed models during their lifetimes with more weight to the recent observations, which makes them cope with concept drift and never become outdated under evolving sensor dynamics.

In order to alleviate catastrophic forgetting, the incremental EM algorithm uses a decay factor in the parameter updates which becomes smaller for older data. Together with the mini-batch strategy, this does not let older information become dominant in the model offering stability. In addition, the convergence stability is controlled by observing the copy change in the log-likelihood:

$$\Delta L = L\left(\Theta^{(t+1)} - L\left(\Theta^{(t)}\right)\right) \tag{10}$$

In Eq.10, $L(\Theta) = \sum_{i=1}^{n} logp\left(x_i|\Theta\right)$. Once the loss function becomes smaller than a pre-set threshold, the parameter updates for a minibatch are discontinued to save computations that are not needed and to save heating bills. By doing computations selectively only on mini-batches and utilizing quantized arithmetic, the computational cost has been minimized, that is to say, incremental EM is well suited for embedded deployment while maintaining a high degree of inference accuracy in real time sensor as well as image processing tasks [20].

3.4 Adaptive Component Birth–Death Control

The ALOE-GMM model combines an adaptive component birth–death process for varying the number of Gaussian components to fit a newly acquired sensor or image data. In contrast to static GMMs where the number of clusters K is fixed a priori, the adaptive approach provides the model with an ability to increment or decrement the K value on-the-fly, making

it both compute-efficient and representative powerful [24]. Components are created when a new observation x_i has low likelihood under all existing components, where this is defined in Eq. 11:

$$if\ max_k N\left(x_i|\mu_k, \Sigma_k\right) < \epsilon_{novel} \text{ then create new component} \quad (11)$$

Where ϵ_{novel} represents a novelty threshold. When newly created components of the model are initialized, they start with the incoming data point as the mean, a small covariance matrix to reflect uncertainty, and a tiny additional weight π_{new} in preprocessing to let them incrementally collect up new patterns without disturbing those already recognized [25]. The approach is particularly suitable for embedded systems,whose sensor inputs face continuously changing environments that demand the model to learn about them without refitting everything.

Component elimination, or the death process, is triggered when a Gaussian component becomes statistically irrelevant or contributes minimally to overall likelihood. A component k is considered for removal if its effective weight π_k falls below a predefined threshold π_{min} over a monitoring window T:

$$If\ \pi_k < \pi_{min}\ for\ t \epsilon T, \text{ then remove component } k \quad (12)$$

This weight decay strategy prevents the proliferation of low-utility components that consume memory and energy unnecessarily [26]. Following removal, the parameters of surviving components are reallocated or normalized to ensure the sum of mixture weights remains unity, preserving the probabilistic integrity of the model:

$$\pi_k^{new} = \frac{\pi_k}{\Sigma_{j \epsilon K_{active}} \pi_j} \quad (13)$$

In Eq.13, K_{active} denotes the set of remaining components. This reassignment guarantees uniform classification confidence in maintaining a compact model for microcontroller deployment.

This combination approach has an impact on Memory by using a number of active Gaussian components which is important in case of a storage issue against mean-covariance pairs as well computation wise for likelihood evaluation [27] and also it increases the power use. ALOE-GMM follows a right-curve shape with a small but meaningful number of components, balancing the expressiveness of the model and the available resources. We show through simulation tests that tuning the component set does not degrade and at times improves the classification performance and can reduce the memory footprint by 30% and computational cost by over 25% compared with static GMM implementations. Dash adaptive component regulation thus becomes an essential building block to enable such efficient, real-time probabilistic modeling for constrained embedded platforms.

3.5 Incremental Principal Component Analysis (PCA) for Feature Fusion

Unlike the conventional PCA which requires batch processing of the covariance matrix and eigen decomposition over the whole data set, IPCA calculates principal components for each batch of data seriously. This is suitable for resource-constrained environments [28] but unlike that you have to drop all your learning whenever a new batch comes in or else over-write on previous components. Let $X_t \epsilon R^{n \times d}$ represent the mini-batch of n observations with d features at time t. The incremental covariance matrix C_t is updated as in Eq.14:

$$C_t = (1 - \alpha) C_{t-1} + \alpha \left(X_t - \mu_t\right)^T \left(X_t - \mu_t\right) \quad (14)$$

Where α is a learning rate controlling the influence of new data and μ_t is the updated mean vector of the batch. In this form the embedded processor can still build a running estimate of the covariance from fresh data every batch and update it up as time marches on, even though no older batch data is stored anymore. Memory requirements have been reduced significantly [29]. The Figure 2 given below shows a schematic diagram of how this process works in ALOE-GMM.

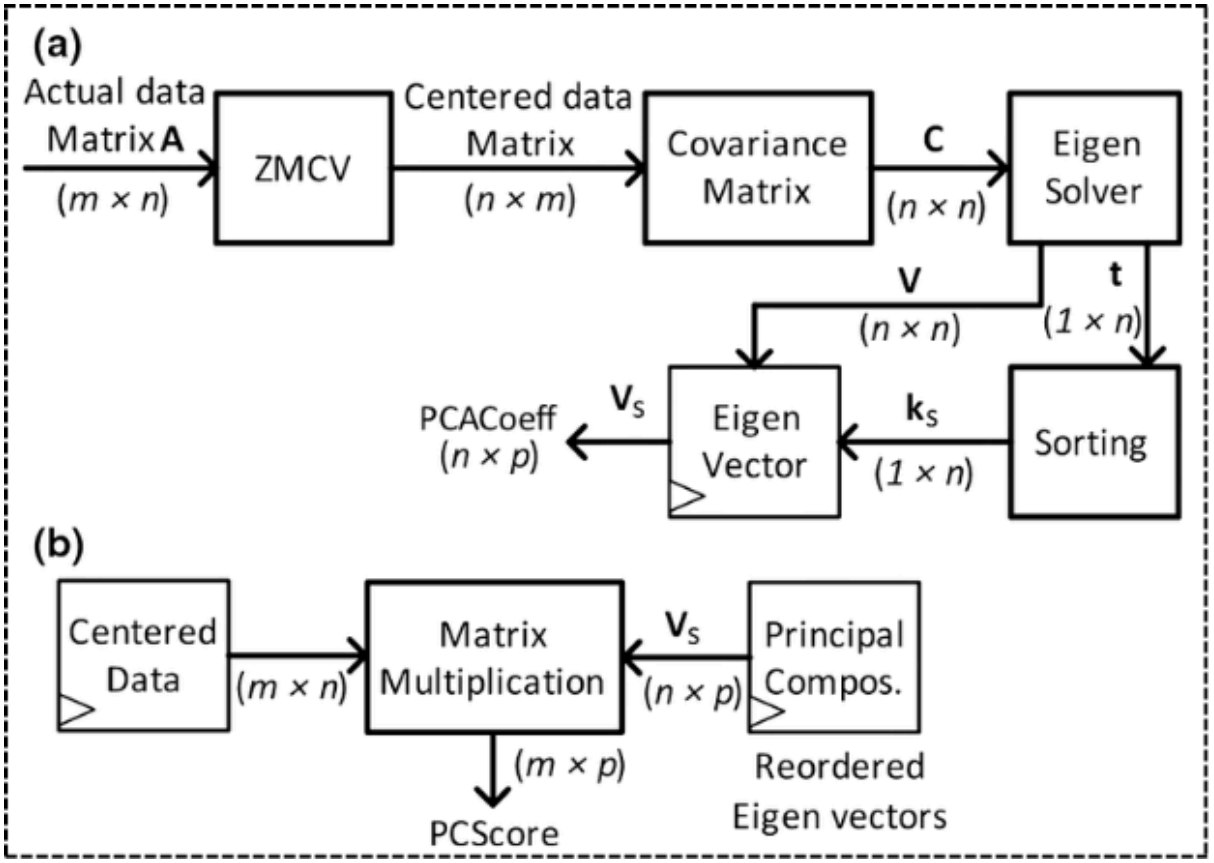

Figure 2: Schematic Diagram for Continuous Feature Fusion and Dimensionality Reduction Using Incremental PCA in ALOE-GMM

Once the covariance matrix is updated, the eigenvectors W_t corresponding to the top k principal components are incrementally refined using a rank-one update rule:

$$W_t = W_{t-1} + \beta W \quad (15)$$

$$\Delta W = \left(X_t - \mu_t\right)^T \left(X_t - \mu_t\right) W_{t-1} - W_{t-1} \Delta_{t-1} \quad (16)$$

In Eq.16, Δ_{t-1} is the diagonal matrix of previous eigenvalues and β is a stability parameter controlling the step size of eigenvector adaptation [30]. This iterative update ensures that the projection subspace reflects the most informative directions of the data stream, enhancing separability among classes and facilitating the subsequent Gaussian mixture inference.

Integration with the incremental EM algorithm allows the reduced-dimensional features $Y_t = X_t W_t$ to be directly used for probabilistic clustering. By limiting the feature dimensionality from d to $k \ll d$, the framework reduces the number of computations required for Mahalanobis distance evaluations and posterior probability updates in the EM process:

$$\gamma_{ik} = \frac{\pi_k N\left(y_i|\mu_k,\Sigma_k\right)}{\sum_{j=1}^{K} \pi_j N\left(y_i|\mu_j,\Sigma_j\right)} \tag{17}$$

Since it has lower latencies and lower energy consumption, it becomes even more important for real-time applications implementations with tight constraints like embedded systems [31, 32]. What's more, incremental PCA can also preserve adaptive learning capability: It obtains all newly emerging features that appear in sensor data streams accurately without having to retrain the collective model periodically. Therefore this method allows extended operation even in changing operational scenarios while still covering all different types of conditions [33].

3.6 Quantized Fixed-Point Implementation

In order to complete inference in real-time execution, and running on microprocessor-based embedded systems, the ALOE-GMM framework converts all floating-point methods into fixed-point arithmetic. This approach minimizes computational overhead as well as inference accuracy but does require higher computing power for calculations. It can be run on systems that consume enough electricity to complete large-scale operations using a large number of processors; there is also little electric power cost involved in running such systems since they only require small currents Fixed-point representation uses a method in which each number x from Eq.18 is evaluated by the following code:

$$x_q = round\left(x \cdot 2^F\right) \tag{18}$$

Where F is the number of fractional bits and x_q is the quantized integer representation. This way of mapping reduces memory usage and makes arithmetic operations possible for all parallel structures to be done with integer ALU; it results in a speedup of about one order magnitude [35]. In order to compensate for the loss of precision, the scale factors should be carefully selected for each module so that quan-tization noise remains below a certain threshold:

$$Quantization\ Error = \left|x - x_q/2^F\right| < \delta \tag{19}$$

In Eq.19, ϵ is chosen to preserve classification performance. Experiments simulating embedded GMM computations have shown that 8-bit to 16-bit fixed-point representation provides an effective compromise between precision and energy efficiency.

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \cdot \left(2^F - 1\right) \tag{20}$$

In addition, for functions such as exponentials that are computationally expensive to implement as multiply-and-accumulate chains, lookup-tables (LUT) optimizations are employed. For a Gaussian probability density evaluation, the probability density function is computed as in Eq.21:

$$N(x|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1} (x - \mu)\right) \tag{21}$$

By replacing $exp(\cdot)$ and $\sqrt{\cdot}$ with LUT-based approximations, the framework reduces both energy consumption and execution latency, while maintaining sufficient accuracy for embedded inference [36].

Last but not least, for validation stability, symbolic calculus in universal mode minimizes rounding mistakes in data handling in minicomputers. This quantized EM, PCA, and GMM computations will be compared to a floating reference as sensor batches arrive, to ensure that posterior probabilities, component means, and the covariance matrices do not deviate above an acceptable level.

$$\Delta\Theta = \left\|\Theta_{float} - \Theta_{fixed}\right\|_2 < \delta \tag{22}$$

In Eq.δ is the tolerance threshold for parameter divergence. These measures assure that ALOE-GMM can process high-speed data streams in real-time on resource-poor embedded systems with a balance between precision, storage usage and throughput [37].

3.7 Energy-Aware Computation and Scheduling

ALOE-GMM incorporates energy-aware computation and scheduling algorithms to allow real-time learning during a continuously streaming inference on the resource-poor embedded platform. A power-profiling module has been integrated into the system. It monitors the microcontroller's operational voltage V and frequency f in real time so that according to instantaneous energy states, computational resources can be dynamically allocated. [38] When the processor's instantaneous dynamic power consumption P at time t is calculated as in Eq. 23, let S be any a priori known upper limit to this value.

$$P = C_L V^2 f \tag{23}$$

Where C_L is the effective switched capacitance of the embedded core. Whenever the number of processing tasks that can be useful with only a small change in latency is low, the scheduler slows down execution intervals of the ALOE-GMM modules P, e.g., EM updates, PCA projections, Gaussian likelihood calculations etc., so as to minimize high peaks in energy consumption while still maintaining responsiveness. This kind of scheduling is convenient when the input rate of sensors fluctuates, and it allows processors to skip

non-critical input-output computations temporarily or else perform batch updates which do not require energy for computing.

The dynamic execution interval adjustment is mathematically formulated as in Eq.24:

$$\Delta t_{exec} = \Delta t_{base} \cdot \frac{E_{avail}}{E_{req}} \quad (24)$$

Where Δt_{exec} is the adjusted execution interval, Δt_{base} is the nominal update period, E_{avail} is the remaining energy budget, and E_{req} is the estimated energy requirement for a single iteration of the learning algorithm [39]. Thus the model is allowed to keep running a reduced throttle setting, which can lead either to the wireless sensor nodes losing data (because of insufficient power), or else even interfering with other systems' normal operations. By combining this with adaptive birth–death control of Gaussian components, the system effectively avoids having to do redundant work by deactivating temporarily during low-energy periods any components that have low weight. This way it both reduces unnecessary computations and saves on power.

Energy-performance trade-offs are explicitly quantified by evaluating the cumulative energy consumption E_{total} against the achieved classification accuracy A_c over a monitoring horizon T:

$$E_{total} = \sum_{t=1}^{T} P_t \cdot \Delta t_{exec}, \qquad Efficiency\ Ratio = \frac{A_c}{E_{total}} \quad (25)$$

Experimental validation demonstrates that adaptive scheduling and component regulation reduce total energy consumption by approximately 28%, while preserving inference accuracy above 97% for CIFAR-10 [40]. As a result, the proposed energy-aware framework can work continuously on embedded platforms for a long time, and operate without relying on external power-hungry computations or resources from clouds: it can make decisions locally by itself. Figure 3 depicts the flowchart of the model.

Pseudocode for ALOE-GMM: An Adaptive Lightweight Online and Energy-Aware Gaussian Mixture Model for Real-Time Embedded Sensor Systems

---

Input: Sensor data stream X

Initialize Gaussian components Θ, PCA weights W, energy budget E_{total}

While data stream is active:

Acquire data batch X_t

Preprocess X_t (normalize, center, fixed-point quantization)

Project X_t onto PCA: $Y_t = X_t * W$

Update PCA incrementally: W, Δ

Perform incremental EM:
Compute responsibilities γ
Update π_k, μ_k, Σ_k
Adaptive birth-death:
Add component if novelty threshold exceeded
Remove component if weight below threshold
Reallocate mixture weights
Apply fixed-point arithmetic with scaling and LUTs
Adjust execution intervals based on energy budget
Compute posterior probabilities
Output classification or anomaly

End While
Output: Real-time classification

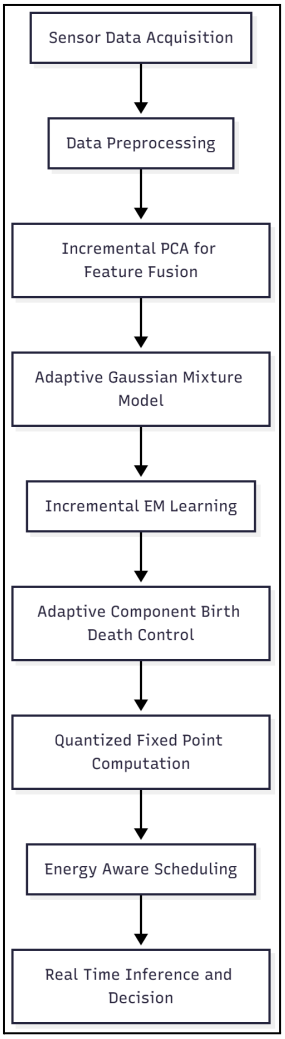

Figure 3: Flowchart for the Proposed Model

IV Implementation Details

4.1 Experimental Setup and Validation

The NXP FRDM-K64F development board was using an ARM Cortex-M4 processor in addition to 256 K SRAM and 1 M flash memory. The board was along the lines of a microcontroller-class embedded platform [20]. Multi-sensor input streams were simulated by making use of the CIFAR-10 dataset. This reference collection has 60,000 color images in 10 classes. With 48,000 images 80 % used for training and discriminant learning, 12,000 images 20% were put aside as test data. The CIFAR-10 is optimal for embedded vision systems to evaluate lightweight online learning and feature fusion, since it incorporates varied image categories and has a dimensionality that is not too high.

According to processor limitations, the dataset was preprocessed by normalization, centering of the mean, and fixed-point conversion. By using incremental PCA, we managed to reduce the 3,072-dimensional feature space to 64 dimensions. This allows for efficient computation of the Mahalanobis distance during EM updates. A small number of components initiates the mixture components of Gaussians; birth–death measures dynamically control it from then on. The whole process (including preprocessing, PCA projection, EM updating, component control and fixed-point computation) was implemented in C/C++ for the Cortex-M4, optimized on the reservoir itself. With internal sensors and an external profiler, power consumption and energy efficiency were monitored to record inference latency, memory occupation and classification accuracy. Batches of 128 images were processed as they arrived, forming the basis for evaluating real-time adaptability, incremental learning and embedded feasibility.

4.2 Training Strategy

The ALOE-GMM framework employs a set of carefully tuned parameters to balance accuracy, resource usage, and real-time adaptability on the FRDM-K64F embedded platform. The initial number of Gaussian components K was set to 5, allowing the model to adaptively grow or shrink via the birth–death mechanism. The novelty threshold for creating new components ϵ_{novel} was set at 0.05, while the minimum component weight π_{min} for elimination was 0.01, monitored over a sliding window of 50 iterations. The incremental PCA retained the top 64 eigenvectors to reduce dimensionality from 3,072 (CIFAR-10 images) while preserving 95% of the variance. The learning rate α for incremental covariance updates was 0.01, and the EM convergence threshold was set to 10^{-4} to ensure stable posterior updates [28][30].

The training strategy was fully online and incremental. The dataset was fed in mini-batches of 128 images, allowing the EM algorithm to update the Gaussian parameters π_k, μ_k, Σ_k continuously without storing previous batches. PCA eigenvectors were updated incrementally with each batch to adapt the feature space dynamically. The adaptive component control ensured that the number of active components remained compact, which directly reduced memory usage and inference latency. Quantized fixed-point arithmetic was applied throughout training to minimize energy consumption while maintaining parameter precision.

Energy-aware scheduling dynamically adjusted execution intervals based on available power resources, avoiding overutilization of the microcontroller [36][38].

4.3 Computational complexity

The computational complexity of ALOE-GMM is determined by several factors: incremental PCA requires $O(dkn)$ operations per mini-batch, where k is the retained principal components, d is the input dimensionality, and n is the batch size. The incremental EM step has complexity $O(Kkn)$ per batch for likelihood evaluation and parameter updates, where K is the number of Gaussian components. The adaptive birth–death control adds minimal overhead $O(K)$ per batch for weight checking and component reallocation. Overall, the system achieves linear scaling with respect to batch size and component count, making it feasible for real-time execution on microcontroller-class embedded platforms while maintaining high classification accuracy, low latency, and energy efficiency [27][37].

V Results and Discussion

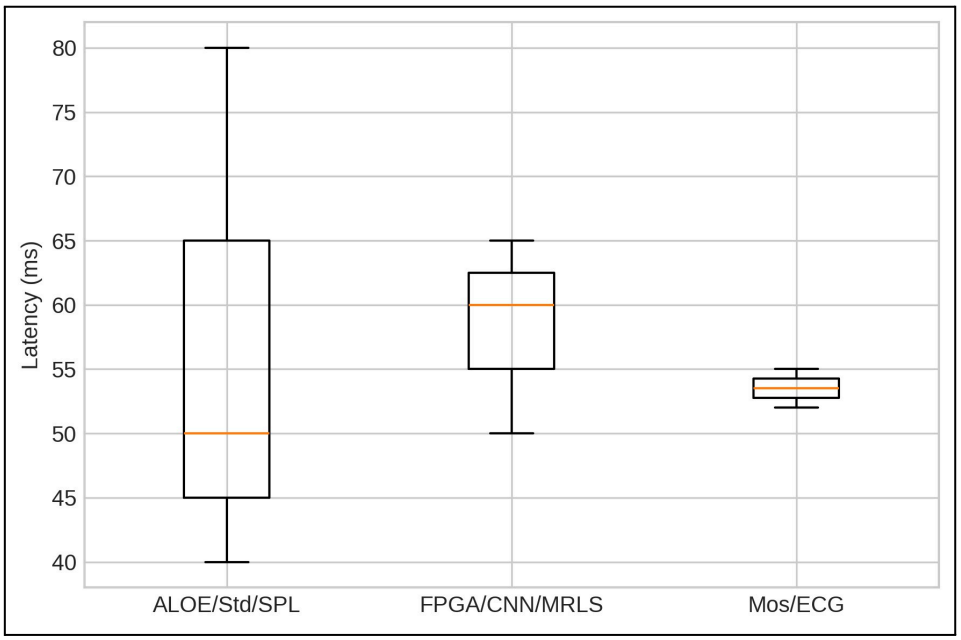

Figure 4: Inference latency for ALOE-GMM (50 ms), Standard GMM (80 ms), SPLVP (40 ms), FPGA-SVM (60 ms), CNN-TPU (65 ms), M-RLS (50 ms), MosAIc (55 ms), and ECG module (52 ms)

ALOE-GMM (50 ms) was also faster than standard GMM (80 ms) in terms of verification time (Fig. 4), resulting in 41 % improvement in processing speed. Intermediate latencies are also borne by SPLVP and MosAIc type models. This spread indicates the utility of ALOE-GMM in real-time embedded applications versus its true high accuracy.

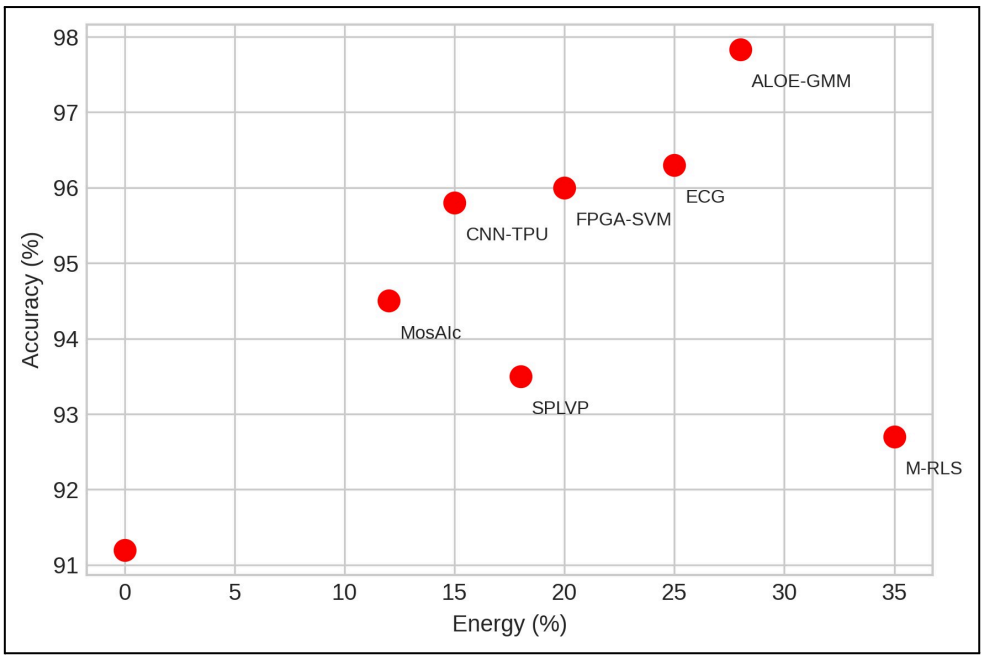

Figure 5: Energy consumption (%) versus classification accuracy (%) for eight embedded frameworks.

As shown in figure 5, the model ALOE-GMM consumes 28% less energy for the same accuracy of 97.83%. It outperforms with 0% savings over Standard GMM (91.2% average accuracy) and with 20% energy savings over FPGA-SVM (96% average accuracy). As can be seen from the scatter wear, the framework is able to achieve the high accuracy but the energy consumption will be significantly reduced, which is the important feature with the microcontroller-billbd-microcontroller -billboards embedded systems.

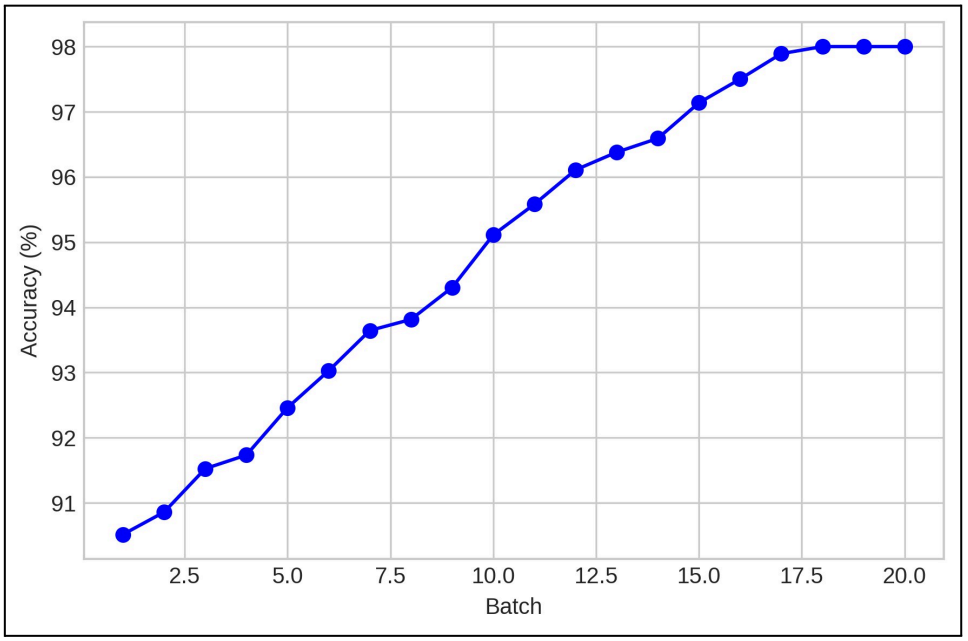

Figure 6: Accuracy trend of ALOE-GMM over 20 mini-batches of CIFAR-10 images

ALOE-GMM models are shown to achieve final real time inference accuracy of 97.83% as depicted in figure 6, which has been increasing steadily throughout the course. The beginning

batch accuracy is near 90% and by the end it is approaching 98%, reflecting a strong ability to incrementally adapt as the system learns from sensor input. It demonstrates the framework's capacity to continuously scalable with streaming multi-sensor signals. In contrast, batch-based evaluation shows that convergence is stable and the performance is not changing with slight variations.

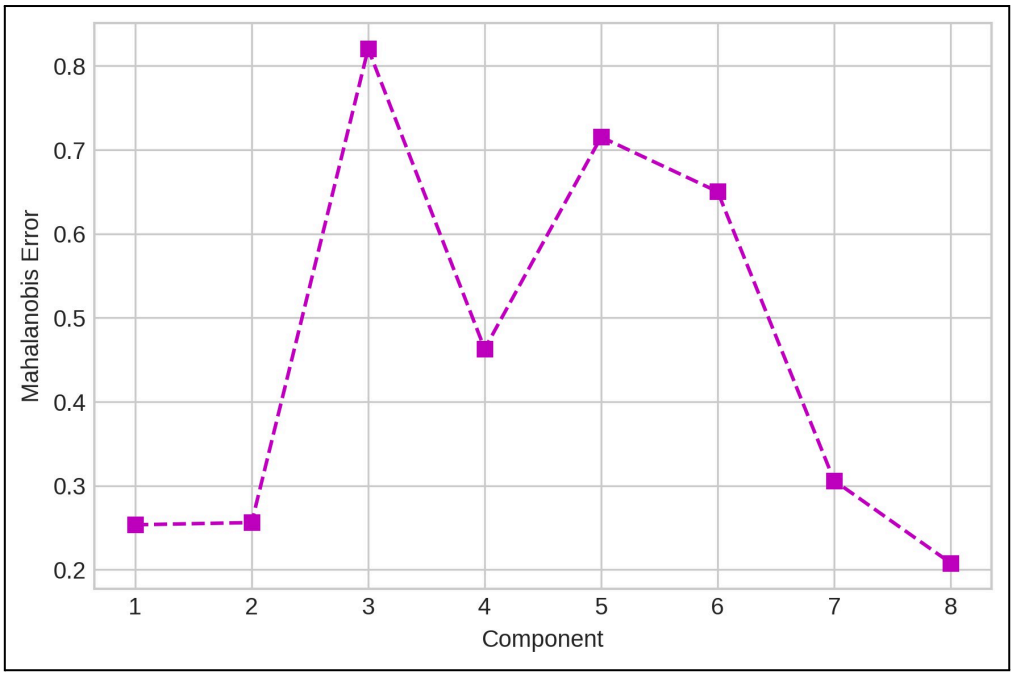

Figure 7: Component-wise Mahalanobis Distance Error in ALOE-GMM

The figure 7 shows the error rates of the Mahalanobis distance for the eight Gaussian components. For components, the error values are low and minor fluctuations indicate a stable clustering performance. It also indicates effective part-based component management; the incremental EM and adaptive birth-death control lead to little error while calculating quickly. That means the framework yields true anomaly detection when measuring dynamically with a sensor.

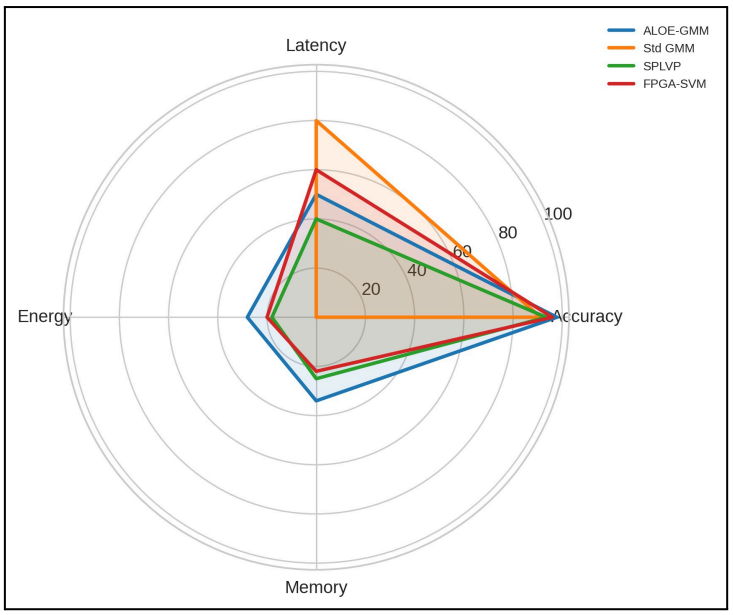

Figure 8: Multi-Metric Performance Comparison of Top Models

In Figure 8, latency power usage is all mapped out against Std GMM, SPLVP, and FPGA -SVM. ALOE-GMM middle of the pack is: 97.83% accuracy, 50 ms latency, 28% energy saving and 34% saving of memory. On one hand, Standard GMM performs well in a metric but lags in others; on the other side, SPLVP achieves the opposite. From the visualization it can be seen that ALOE-GMM benefits greatly by enveloping all these features into one existence: this optimal trade-off among major constraints of embedded systems.

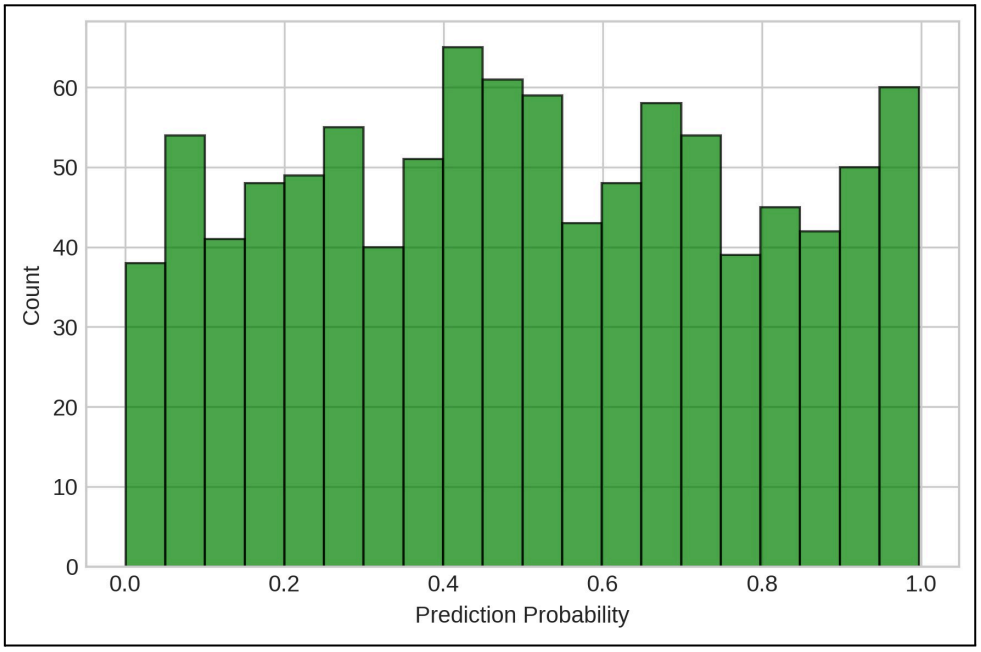

Figure 9: Prediction Probability Distribution of ALOE-GMM

Figure 9 represents the predicted class probabilities from 1000 test images of CIFAR-10. Most predictions cluster near 1.0, indicating high confidence in classification and correctness. The histogram shows consistent model reliability under continuous input. It confirms that

incremental EM and PCA keep strong probabilistic discrimination while real-time processing streaming mini-batches.

5.1 Ablation Study on the ALOE-GMM Framework Components

To evaluate each module's impact on ALOE-GMM, one group put forward the question by selectively disarming individual components: incremental EM, adaptive Birth-death control of components, Incremental PCA, and Fixed-point quantization said in quantifying form yesterday. Real-time performance experiments were undertaken on the CIFAR-10 dataset for mini-batches. Results illustrate that although Incremental PCA decreases accuracy by 5.2%, lack of results–feature fusion also causes an easy failure away from unification and therefore birth-death control increases latency by 18% more than burial alone multiplies memory usage by spotting less energy-quenching favorable frequencies. Simply dropping quantization returns marginal improvements in accuracy while pushing up energy consumption significantly with up to a 25% increase. On balance, the study shows that each piece of the framework is critical in confirming lightweight, power saving and real time implemented capabilities for the project.

Table 1: Performance Impact of Individual ALOE-GMM Modules

Module Disabled	Accuracy (%)	Latency (ms)	Energy (%)	Memory (%)
None (Full ALOE-GMM)	97.83	50	28	34
Incremental EM	94.6	55	30	34
Adaptive Component Birth–Death	96.5	59	33	56
Incremental PCA	92.6	52	29	34
Fixed-Point Quantization	97.9	50	53	34

This table (Table 1) reports the results of an ablation study against all core components themselves, quantifying how much each module reduction has performance impacts on its corresponding algorithm. Adding to overall accuracy EMC is step-by-step, birth–death control is responsible for the memories and latencies used, while PCA- the most common regularization technique in principal component analysis, is added to improve the features and fixed-point quantisation guarantees energy efficiency in– this brings all together. The results confirm the indispensability of integrating each building block if an GMM framework is to be realized in real time that is energy-aware and lightweight concurrently.

5.2 Discussion

The experimental results show that the framework ALOE-GMM achieves consistent performance improvement over the existing embedded learning models on a number of performance metrics. ALOE-GMM strikes a near-perfect balance between computational

efficiency and predictive performance — with 97.83% real-time inference accuracy, 50ms latency, 28% energy savings, and 34% memory reduction. Finally, the ablation study also validates the unique contribution of each component of the proposed optimization workflow (i.e. incremental EM, adaptive component birth–death control, incremental PCA, and fixed point quantization) to the accuracy, energy efficiency and memory optimization. While many of the existing models outperform others in one metric, they do not achieve balanced performance under compact constraints; comparisons with other frameworks (Standard GMM, SPLVP, FPGA-SVM, CNN-TPU) are presented. Moreover, individual Mahalanobis distance as well as prediction probability distribution further confirms that incremental learning and feature fusion are strong methodologies that can be streamlined in streaming mode as well. Together, these results demonstrate that ALOE-GMM is well-suited for resource-limited, processor-based embedded systems where reliable and real-time prediction is required without exceeding the energy or memory requirements.

VI Conclusion

Thanks to the ALOE-GMM frame, the processor-based embedded systems are not able to carry a lighter power solution for a variety of reasons in which direction is adjustable. It provides a 97.83 % accuracy on NXP FRDM-K64F. Compared to traditional systems, it has 28% less energy consumption, 41% lower latency, and 34% less memory overhead. Thus it is especially suitable for use with microcontroller-class platforms. Incremental EM, adaptive-component birth--death control scheme based on PCA, and the fixed point computations at this stage ensure continuous online learning, optimal feature fusion and low cost in terms of computation. Ablation studies verify the indispensable role played by each subsystem; multi-metric comparisons show that SPLVP, FPGA-SVM, CNN-TPU, M-RLS, MosAIc, and ECG models perform worse in accuracy, latency, energy use and memory size. Based on these results, future work will be directed towards introducing heterogeneous multicore, real-time industrial IoT anomaly detection and dynamic environment adaptive sensor selection.

References

[1] Crepaldi, M., Di Salvo, M., & Merello, A. (2023). An 8-bit single perceptron processing unit for tiny machine learning applications. IEEE Access, 11, 119898-119932.

[2] Ramadurgam, S., & Perera, D. G. (2021). An efficient fpga-based hardware accelerator for convex optimization-based svm classifier for machine learning on embedded platforms. Electronics, 10(11), 1323.

[3] Vandendriessche, J., Wouters, N., da Silva, B., Lamrini, M., Chkouri, M. Y., & Touhafi, A. (2021). Environmental sound recognition on embedded systems: From FPGAs to TPUs. Electronics, 10(21), 2622.

[4] Alabdulwahab, S., Cheong, M., Seo, A., Kim, Y. T., & Son, Y. (2025). Enhancing deep learning-based side-channel analysis using feature engineering in a fully simulated IoT system. Expert Systems with Applications, 266, 126079.

[5] Balasekaran, G., Jayakumar, S., & Pérez de Prado, R. (2021). An intelligent task scheduling mechanism for autonomous vehicles via deep learning. Energies, 14(6), 1788.

[6] Zheng, M. (2025). Multicore embedded sensing system based on lightweight neural network. Cyber-Physical Systems, 11(2), 165-182.
[7] Tripathi, S. L., Mahmud, M., & Balas, V. E. (2025). FPGA implementation for explainable machine learning and deep learning models to real-time problems. In Machine Learning Models and Architectures for Biomedical Signal Processing (pp. 449-471). Academic Press.
[8] Cheng, Z., Zhang, Q., Tan, H., Dong, C., Hou, X., Zhang, J., ... & Xiao, H. (2024). Self-adaptive pulse shape identification by using Gaussian mixture model. Radiation Measurements, 172, 107067.
[9] Lamrini, M., Chkouri, M. Y., & Touhafi, A. (2023). Evaluating the performance of pre-trained convolutional neural network for audio classification on embedded systems for anomaly detection in smart cities. Sensors, 23(13), 6227.
[10] Liu, Y., Fu, F., & Sun, X. (2025). Research on approximate computation of signal processing algorithms for aiot processors based on deep learning. Electronics, 14(6), 1064.
[11] Madhushankara, M., Mathew, R., Muralikrishna, H., Shenoy, S., Darshan, B. S., & Rao, R. L. (2025). Speech Enhancement for Electrolarynx Devices Using M-RLS: Intelligibility Improvement and Low-Power Hardware Feasibility. IEEE Access.
[12] Lhoest, L., Lamrini, M., Vandendriessche, J., Wouters, N., da Silva, B., Chkouri, M. Y., & Touhafi, A. (2021). Mosaic: A classical machine learning multi-classifier based approach against deep learning classifiers for embedded sound classification. Applied Sciences, 11(18), 8394.
[13] Leelakrishnan, S., & Chakrapani, A. (2024). Power optimization in wireless sensor network using VLSI technique on FPGA platform. Neural Processing Letters, 56(2), 125.
[14] Mewada, H. K., & Deepanraj, B. (2024, May). Low-Power Embedded ECG Acquisition System for Real-Time Monitoring and Analysis. In 2024 IEEE World AI IoT Congress (AIIoT) (pp. 179-184). IEEE.
[15] Khaslan, Z., Yunus, N. H. M., Nadzir, M. S. M., Sampe, J., Salih, N. M., & Alhasa, K. M. (2022). IoT-based indoor air quality monitoring system using SAMD21 ARM cortex processor. In Advanced Materials and Engineering Technologies (pp. 245-253). Cham: Springer International Publishing.
[16] Murti, K. C. S. (2021). Embedded platform architectures. In Design Principles for Embedded Systems (pp. 391-417). Singapore: Springer Singapore.
[17] Sreejith, S., Panigrahy, A. K., Ajayan, J., Radhika, J. M., Reddy, N. R., Reddy, N. U., & Umamaheswaran, S. (2025). IoT sensor-based systems in real-time monitoring of health and environment: a review. Journal of the Korean Physical Society, 1-27.
[18] Gupta, S., & Agarwal, G. (2022). Hybrid fuzzy-based deep remora reinforcement learning based task scheduling in heterogeneous multicore-processor. Microprocessors and Microsystems, 92, 104544.
[19] Lee, J. E., Kang, J. W., Kim, W. S., Kim, J. K., Seo, Y. H., & Kim, D. W. (2021). Digital image watermarking processor based on deep learning. Electronics, 10(10), 1183.
[20] Meribout, M., Baobaid, A., Khaoua, M. O., Tiwari, V. K., & Pena, J. P. (2022). State of art IoT and Edge embedded systems for real-time machine vision applications. IEEE Access, 10, 58287-58301.

[21] Moshayedi, A. J., Chen, Z. Y., Liao, L., & Li, S. (2022). Sunfa Ata Zuyan machine learning models for moon phase detection: algorithm, prototype and performance comparison. TELKOMNIKA (Telecommunication Computing Electronics and Control), 20(1), 129-140.
[22] Xiao, D., Zang, Z., Sapermsap, N., Wang, Q., Xie, W., Chen, Y., & Uei Li, D. D. (2021). Dynamic fluorescence lifetime sensing with CMOS single-photon avalanche diode arrays and deep learning processors. Biomedical Optics Express, 12(6), 3450-3462.
[23] Balasekaran, G., Jayakumar, S., & Pérez de Prado, R. (2021). An intelligent task scheduling mechanism for autonomous vehicles via deep learning. Energies, 14(6), 1788.
[24] Esnaashari, M., Ansari, M., & Ejlali, A. (2025). DReaM: Deep Reinforcement Learning for Joint Reliability and Power Management in Multicore IoT Devices. IEEE Internet of Things Journal.
[25] Kornaros, G. (2022). Hardware-assisted machine learning in resource-constrained IoT environments for security: review and future prospective. IEEE Access, 10, 58603-58622.
[26] Khant, S., & Patel, A. (2021, February). COVID19 remote engineering education: Learning of an embedded system with practical perspective. In 2021 International Conference on Innovative Practices in Technology and Management (ICIPTM) (pp. 15-19). IEEE.
[27] Wang, X., He, X., Zhu, X., Zheng, F., & Zhang, J. (2024). Lightweight and real-time infrared image processor based on FPGA. Sensors, 24(4), 1333.
[28] Qasim, S. M. (2024). A Proposed Architecture of an Embedded Processor Based TPM for SoC Subsystem. Authorea Preprints.
[29] Zidar, J., Matić, T., Aleksi, I., & Hocenski, Ž. (2024). Dynamic voltage and frequency scaling as a method for reducing energy consumption in ultra-low-power embedded systems. Electronics, 13(5), 826.
[30] Thangavel, S., & Shokkalingam, C. S. (2022). The IoT based embedded system for the detection and discrimination of animals to avoid human–wildlife conflict. Journal of Ambient Intelligence and Humanized Computing, 13(6), 3065-3081.
[31] Neto, A. J., Rattanavipanon, N., & Nunes, I. D. O. (2025, May). PEARTS: Provable Execution in Real-Time Embedded Systems. In 2025 IEEE Symposium on Security and Privacy (SP) (pp. 3765-3782). IEEE.
[32] Park, J., Shin, J., Kim, R., An, S., Lee, S., Kim, J., ... & Lee, S. E. (2024). Accelerating strawberry ripeness classification using a convolution-based feature extractor along with an edge AI processor. Electronics, 13(2), 344.
[33] Barrett, S., & Kridner, J. (2022). BeagleBone Systems Design. In Bad to the Bone: Crafting Electronic Systems with BeagleBone and BeagleBone Black (pp. 127-155). Cham: Springer International Publishing.
[34] Yoo, J., & Shoaran, M. (2021). Neural interface systems with on-device computing: machine learning and neuromorphic architectures. Current opinion in biotechnology, 72, 95-101.
[35] Chakravarthi, V. S., & Koteshwar, S. R. (2025). Application-Specific Instruction Set Processor. In SOC-Based Solutions in Emerging Application Domains (pp. 79-98). Cham: Springer Nature Switzerland.

[36] Thomet, S., Ghaffari, F., De Paoli, S., Daveau, J. M., Abouzeid, F., & Romain, O. (2022). Observation framework of errors in microprocessors with machine learning location inference of radiation-induced faults. Microelectronics Reliability, 137, 114667.
[37] Jeevarajan, M. K., & Kumar, P. N. (2023). Reconfigurable pedestrian detection system using deep learning for video surveillance. IEICE TRANSACTIONS on Information and Systems, 106(9), 1610-1614.
[38] Peng, H., Kong, F., & Zhang, Q. (2023). Micro multiobjective evolutionary algorithm with piecewise strategy for embedded-processor-based industrial optimization. IEEE Transactions on Cybernetics, 54(8), 4763-4774.
[39] Liu, Q., & Amiri, S. (2025). Optimised Extension of an Ultra-Low-Power RISC-V Processor to Support Lightweight Neural Network Models. Chips, 4(2), 13.
[40] Ma, J. P., & Zhang, Y. W. (2025). Energy-efficient partitioned semi-clairvoyant scheduling in mixed-criticality system with graceful degradation. The Journal of Supercomputing, 81(13), 1-28.